LifeOS Agent: Implementation Plan and App Flow

Nov 1, 2026 · @Shanaya Mahendran

This guide takes LifeOS Agent from an empty folder to a working chat-controlled planner in 12 weeks, one working layer at a time: data, calendar, agent, domains, automation.

Build strategy

Build the calendar first and the AI second: the agent is only as good as the typed tools it can call, so every tool must work by hand before the LLM touches it.

Principles

- **Vertical slices.** Finish one thing end to end (for example "move an event by chat") before starting the next.
- **Tools before prompts.** Write and test each calendar function without AI, then expose it to the agent.
- **One source of truth.** The Postgres database holds all state; the agent and the UI both go through the same calendar service.
- **Safe by default.** The agent proposes, the app validates, and every change can be undone.
- **Start with 4 domains.** Education, Competitions, Karate and Sleep prove the pattern; the other 8 reuse it.

Definition of done for the MVP

- You can plan and adjust a full week using only chat messages
- Conflicts and protected time (sleep, karate) are caught before saving
- A morning brief and a Sunday review arrive without any manual step
- You have used it for your own week for at least 7 days in a row

Prerequisites and setup

Setup takes about half a day and uses free tiers for everything.

Accounts and tools

- Node.js 20 or later, Git and a GitHub account
- Anthropic API key (Claude API) for the agent
- A hosted Postgres with pgvector: Neon or Supabase (free tier)

- Vercel account for hosting and cron jobs
- Optional later: Google Cloud project for Calendar sync, a push notification service

First commands

```
npx create-next-app@latest lifeos --typescript --tailwind --app --eslint
cd lifeos
npm i prisma @prisma/client next-auth zod @anthropic-ai/sdk
npm i @fullcalendar/react @fullcalendar/daygrid @fullcalendar/timegrid
@fullcalendar/interaction
npx prisma init
```

Environment variables (.env.local)

Variable	Purpose
DATABASE_URL	Postgres connection string
NEXTAUTH_SECRET and NEXTAUTH_URL	Auth.js session signing and base URL
ANTHROPIC_API_KEY	Claude API access, server-side only
CRON_SECRET	Protects the scheduled-job endpoints

Never expose ANTHROPIC_API_KEY to the browser: all agent calls run in server route handlers.

Project structure

The codebase separates what users see (app), what the agent does (lib/agent) and what changes data (lib/calendar), so the agent can never bypass the calendar rules.

```
lifeos/
  app/
    (auth)/login/page.tsx
    (app)/today/page.tsx          home and morning brief
    (app)/calendar/page.tsx       month, week, day views
    (app)/domains/[slug]/page.tsx per-domain page
    (app)/review/page.tsx         weekly review
    (app)/settings/page.tsx
    api/chat/route.ts             chat endpoint (streams agent reply)
    api/events/route.ts           manual calendar CRUD
    api/cron/brief/route.ts       daily brief job
    api/cron/review/route.ts      weekly review job
  components/
    chat/ calendar/ domain/ ui/
```

```
lib/
  calendar/                    events, conflicts, free slots, undo
  agent/
    orchestrator.ts            intent routing
    tools.ts                   tool schemas and handlers
    prompts/                   one prompt file per domain
    memory.ts                  save and recall domain notes
  db.ts                        Prisma client
prisma/schema.prisma
```

Rule: files in lib/agent may only call functions exported by lib/calendar, never Prisma directly for events.

Phase 1: Foundation (weeks 1 to 2)

By the end of week 2 you can log in, see a calendar, and create, drag and delete events by hand, with no AI involved yet.

Week 1: app, auth, database

- [ ] Create the Next.js project and push it to GitHub
- [ ] Provision Postgres (Neon or Supabase) and enable the pgvector extension
- [ ] Write the Prisma schema: User, Domain, Event, Task, Memory, AgentLog
- [ ] Run the first migration and seed 12 domains with colors and a protected flag
- [ ] Add Auth.js with Google or email sign-in; protect all (app) routes
- [ ] Build the app shell: sidebar on desktop, bottom tab bar on mobile
- [ ] Deploy to Vercel so every push has a live preview

Week 2: calendar and calendar service

- [ ] Write lib/calendar functions: createEvent, updateEvent, deleteEvent, listEvents
- [ ] Add findConflicts (overlap check) and findFreeSlots (respects sleep and protected blocks)
- [ ] Expose them through api/events with zod validation
- [ ] Add FullCalendar with month, week and day views; drag and resize call updateEvent
- [ ] Add domain color chips that toggle each layer on or off
- [ ] Unit-test conflicts and free slots with edge cases (midnight, back-to-back, all-day)

Done check: create an event in Karate, drag it onto a Sleep block, and see the conflict warning.

Phase 2: Chat agent (weeks 3 to 4)

By the end of week 4, typing "move karate to 7 PM" changes the calendar, and a wrong change can be undone.

Week 3: orchestrator and tools

- [ ] Define the 8 tools as zod schemas in lib/agent/tools.ts (create_event, move_event, cancel_event, query_schedule, find_free_slots, create_task, plan_backwards, save_memory)
- [ ] Write each tool handler as a thin wrapper over lib/calendar
- [ ] Build api/chat: send the message, today's date, user timezone and tool list to the Claude API
- [ ] Run the tool-calling loop on the server: model asks for a tool, server runs it, result goes back, repeat until a final text reply
- [ ] Write the orchestrator system prompt (see Key code) and keep it in a prompt file
- [ ] Log every message, tool call and result to AgentLog

Week 4: safety and chat UI

- [ ] Add confirmation: tool calls that touch protected time or 5+ events return a preview, not a write
- [ ] Add undo: each AgentLog entry stores the inverse operation; "undo" replays it
- [ ] Build the chat panel with streaming replies and a change summary card under each reply
- [ ] Refresh the calendar view after every agent write
- [ ] Write 20 test commands (move, cancel, query, ambiguous, conflicting) and check each result

Done check: 10 real commands from your own week work end to end, and "undo" reverts the last one.

Phase 3: First four domains (weeks 5 to 7)

By the end of week 7, Education, Competitions, Karate and Sleep each work as their own agent instance with their own goals, memory and page.

Week 5: the domain template

- [ ] Define a domain config: name, color, goals, protected flag, default event types, prompt file
- [ ] Build the domain page: goals, task list, notes, mini calendar filtered to that domain

- [ ] Add per-domain memory: save_memory stores a note with an embedding; recall fetches the top 5 matches
- [ ] Make the orchestrator route by domain, passing that domain's prompt and memory to the call

Week 6: Education and Competitions

- [ ] Education: exam countdowns, revision block generator, assignment due dates
- [ ] Competitions: deadline tracker, team task list, pitch-prep blocks before a submission date
- [ ] Add the plan_backwards tool: given a deadline and an estimate, place work blocks in free slots

Week 7: Karate and Sleep

- [ ] Karate: recurring sessions, conditioning slots, grading dates, protected by default
- [ ] Sleep: target bedtime and wake time, wind-down reminder, warning when a change cuts sleep below your minimum
- [ ] Add the load check: warn when a day has more than your set number of planned hours
- [ ] Use it for your own real week and write down every wrong or annoying result

Done check: one full week of your real schedule runs through the app with no manual workarounds.

Phase 4: Smart planning (weeks 8 to 9)

By the end of week 9 the app plans ahead on its own: deadlines turn into work blocks, and a brief and a review arrive without you asking. This is the MVP.

Week 8: auto-scheduling

- [ ] Turn plan_backwards into a background action triggered whenever a deadline or exam is created
- [ ] Rank free slots by rules: avoid protected time, prefer your focus hours, keep buffer between blocks
- [ ] Show auto-created blocks as "suggested" (dashed) until you accept them
- [ ] Add a "reschedule missed blocks" action that moves unfinished work to the next free slots

Week 9: briefs, reviews and notifications

- [ ] Cron endpoint api/cron/brief: build the day's plan, top 3 tasks, and warnings; save to the Today page

- [ ] Cron endpoint api/cron/review (Sunday evening): planned versus done per domain, slipped items, proposed next week
- [ ] Protect both endpoints with CRON_SECRET and schedule them in vercel.json
- [ ] Add web push or email reminders for deadlines, bedtime and karate
- [ ] Add a one-tap "accept next week" button that writes the proposed plan

Done check: create a deadline 10 days away and see revision or work blocks appear, then receive a morning brief the next day.

Phase 5: Full rollout and polish (weeks 10 to 12)

By the end of week 12 all 12 domains are live, Google Calendar is synced, and the app is fast and stable enough to rely on daily.

Weeks 10 and 11: the other 8 domains

Each new domain reuses the template from Phase 3, so the work per domain is a config, a prompt file and any domain-specific action.

Domain	Domain-specific work
Career	Application tracker, CV update reminders, interview prep blocks
Volunteering	Shift events, hours logged per month
Open source	Issue list, PR status, weekly contribution slot
Food	Meal windows, prep slots, a simple weekly plan
Blogging	Idea to draft to publish pipeline, writing slots
Learning	Skill roadmap, daily learning block, project practice
Forward planning	Monthly goals, yearly direction, review prompts
Rest and buffer	Protected downtime, automatic buffer between blocks

Week 12: sync, polish, hardening

- [ ] Google Calendar two-way sync (OAuth, then push your events and read busy times)
- [ ] Rate limits on api/chat and a monthly token budget with a warning
- [ ] Data export (JSON) and account delete
- [ ] Loading, empty and error states on every screen; keyboard shortcuts for the chat box
- [ ] Performance pass: index Event by user, start and domain; paginate AgentLog

- [ ] Optional: add the suggested extras (finance, admin, social, portfolio, travel, habits) as new domains

Done check: all 12 domains appear in the calendar and chat, and a full week runs on the app alone.

Key code

Three pieces carry the system: the schema, the tool definitions and the orchestrator prompt. Treat the snippets below as starting points to adapt.

1. Prisma schema (core models)

```
model Domain {
  id          String  @id @default(cuid())
  userId      String
  slug        String
  name        String
  color       String
  goals       String?
  isProtected Boolean @default(false)
  events      Event[]
  tasks       Task[]
  memories    Memory[]
  @@unique([userId, slug])
}

model Event {
  id        String   @id @default(cuid())
  domainId  String
  title     String
  start     DateTime
  end       DateTime
  status    String   @default("confirmed") // confirmed | suggested |
cancelled
  source    String   @default("manual")    // manual | chat | auto
  priority  Int      @default(2)
  domain    Domain   @relation(fields: [domainId], references: [id])
  @@index([domainId, start])
}

model AgentLog {
  id        String   @id @default(cuid())
  userId    String
  message   String
```

```
  toolCalls Json
  inverse   Json?    // how to undo this change
  createdAt DateTime @default(now())
}
```

2. Tool definition (one example)

```
import { z } from "zod";

export const moveEvent = {
  name: "move_event",
  description: "Move an existing event to a new start time. Keeps its
duration.",
  schema: z.object({
    eventId: z.string(),
    newStart: z.string().describe("ISO 8601 in the user's timezone"),
  }),
  handler: async ({ eventId, newStart }, ctx) => {
    const conflicts = await findConflicts(ctx.userId, eventId, newStart);
    if (conflicts.protected.length) return { needsConfirmation: true,
conflicts };
    return await updateEvent(ctx.userId, eventId, { start: newStart });
  },
};
```

3. Orchestrator system prompt (draft)

```
You are LifeOS, a personal planning agent for one user.
Today is {{date}}. The user's timezone is {{timezone}}.
You can only change the calendar by calling tools. Never claim a change
happened unless a tool result confirms it.
Rules:
1. Find the right event with query_schedule before moving or cancelling.
2. If a request is ambiguous (two matching events), ask one short question.
3. Never overwrite sleep, meals or karate without explicit confirmation.
4. If a tool returns needsConfirmation, describe the conflict and ask.
5. After a change, reply in one or two sentences: what changed and when.
Domains: {{domain_list}}. Relevant notes: {{domain_memory}}.
```

Testing and deployment

Test the calendar rules heavily and the agent with a fixed set of example commands, because the rules are exact and the agent is not.

Layer	What to test	How
Calendar service	Overlaps, midnight spans, time zones, daylight saving, free-slot search	Unit tests with Vitest
API routes	Validation, auth, ownership of every record	Integration tests against a test database
Agent	30 to 50 saved commands with expected tool calls and outcomes	A replay script that runs each command and checks the resulting calendar
Safety	Protected-time edits, bulk changes, undo, prompt injection in event titles	Dedicated cases that must always ask or refuse
UI	Drag, resize, layer toggles, chat on a small phone screen	Playwright end-to-end tests plus manual phone checks

Environments

- **Local:** Docker Postgres or a Neon dev branch
- **Preview:** every pull request on Vercel with its own database branch
- **Production:** Vercel with daily database backups

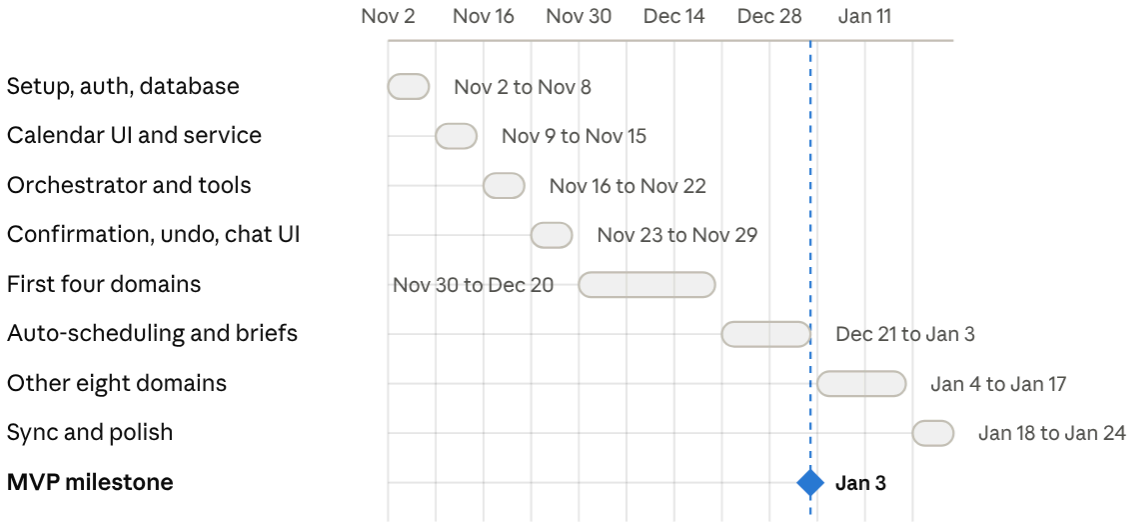
Launch checklist

- [ ] All secrets set in Vercel, none in the repository
- [ ] Cron endpoints reject requests without CRON_SECRET
- [ ] Rate limit and monthly token budget active
- [ ] Every query filters by the logged-in user
- [ ] Undo tested on each tool
- [ ] Backup restored once to prove it works

Roadmap

The MVP lands on Jan 3; the full system follows by Jan 24

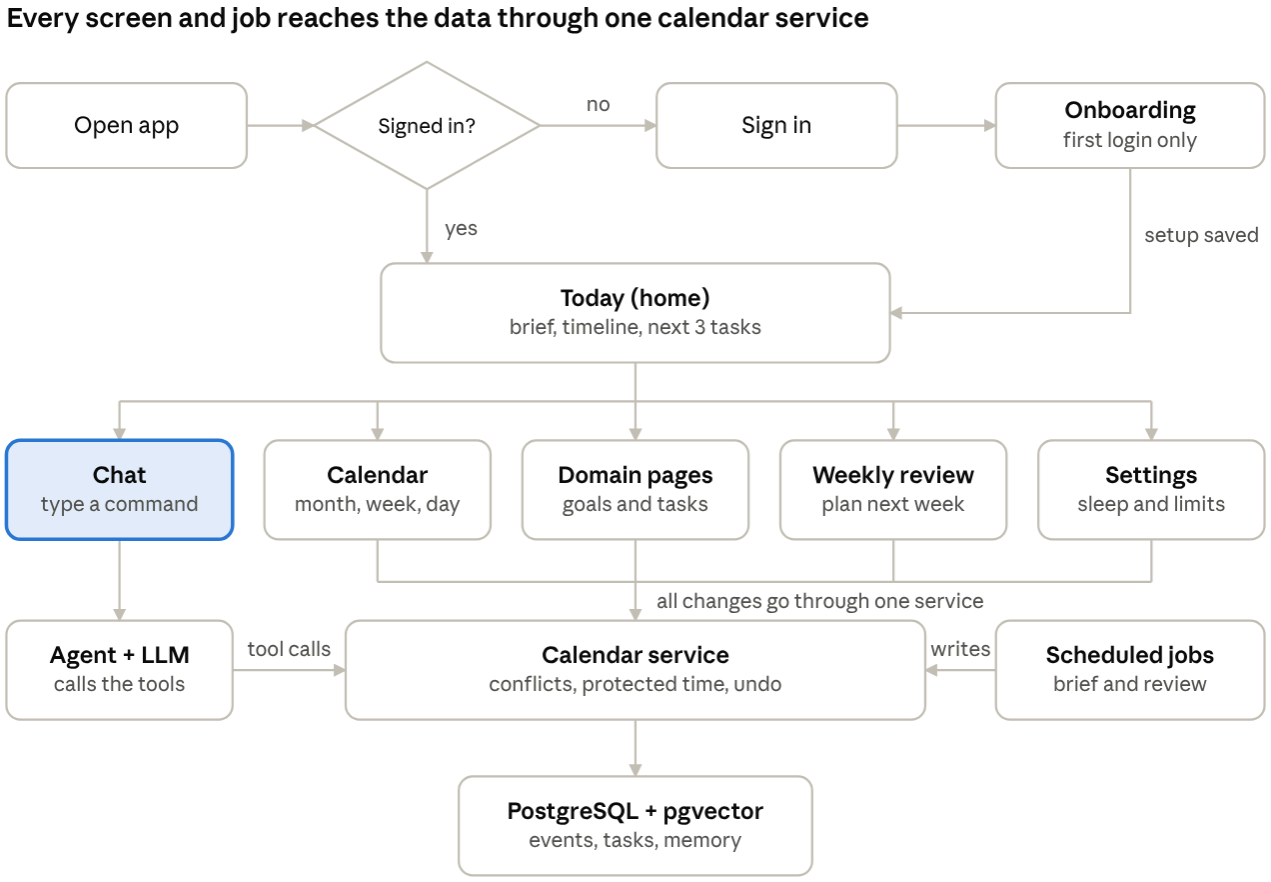
12-week build starting Mon Nov 2, 2026 (assumed start date)



build timeline · 8 work streams, 1 milestone

Weeks run Monday to Sunday from Nov 2, 2026, an assumed start date; shift every date by the same amount if you start later.

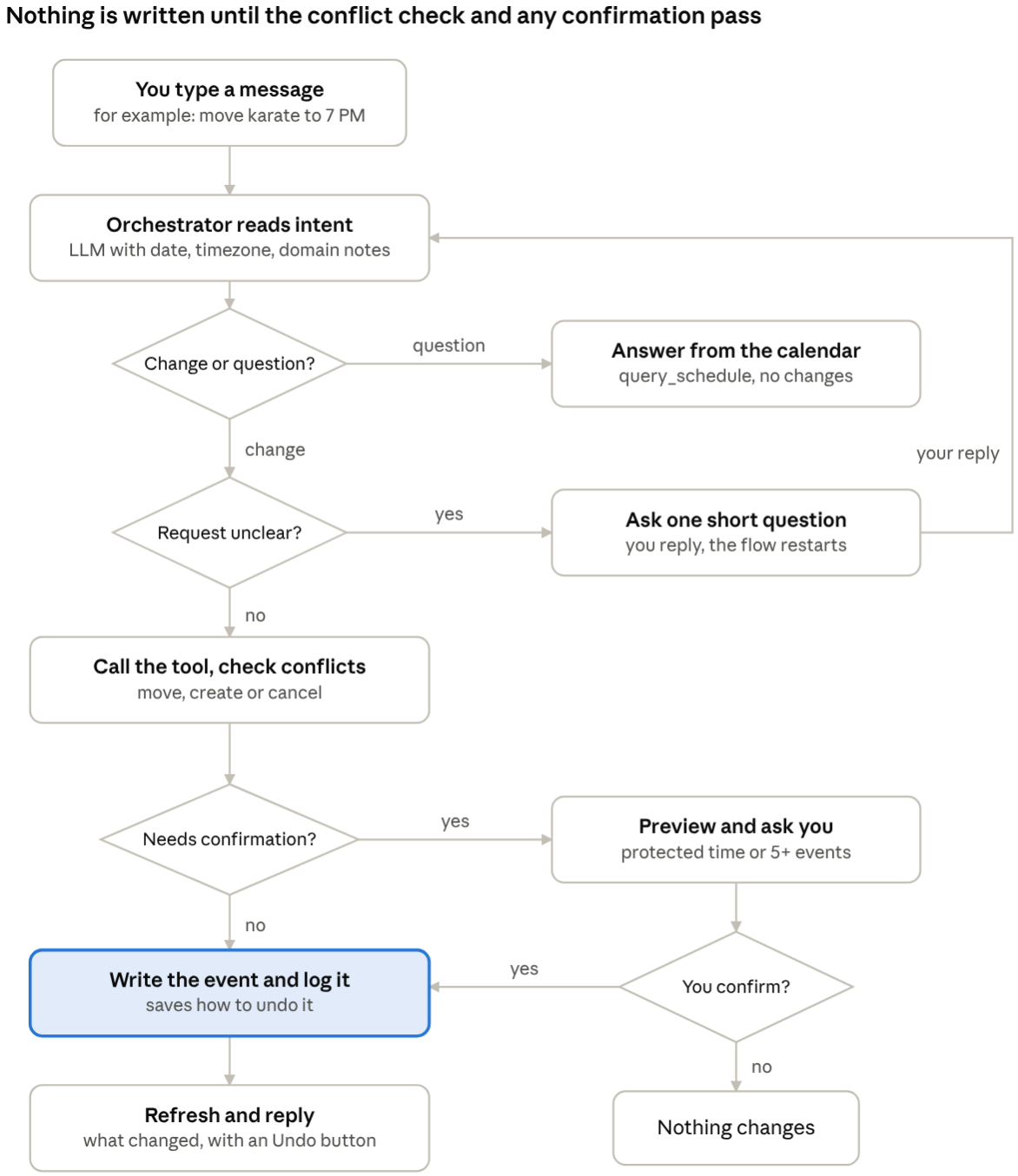
App flow: whole system



system flow · sign-in, 5 screens, 1 service, 1 database

A user signs in, completes onboarding once, and lands on Today. Chat is the only screen that goes through the agent; the other four screens call the calendar service directly, and scheduled jobs write briefs and reviews the same way.

App flow: chat command



chat command flow · 4 decisions, 1 write

Protected time means sleep, meals and karate, and a bulk change means 5 or more events at once. Every write stores its inverse, so Undo restores the previous state.

App flow: automations

Three background flows keep the plan alive without a chat message: auto-scheduling, the morning brief and the weekly review.

Auto-scheduling (runs when a deadline or exam is saved)

1. The app saves the deadline with its domain and a work estimate (for example 12 hours)
2. plan_backwards splits the estimate into blocks of 60 to 120 minutes
3. find_free_slots lists open times before the deadline, skipping sleep, meals, karate and other protected blocks
4. Blocks are placed in the best slots and saved with status "suggested"
5. The Today screen and chat show "I added 6 blocks for your exam" with Accept all and Edit buttons
6. Accepted blocks become confirmed; unfinished blocks can be rescheduled later

Morning brief (every day at your wake time plus 30 minutes)

1. The cron job calls api/cron/brief with the secret key
2. It reads today's events, due tasks and tomorrow's first commitments
3. It checks load (planned hours) and sleep the night before
4. The agent writes a short brief: the plan, the top 3 tasks, any warnings
5. The brief is saved to the Today page and sent as a push or email

Weekly review (Sunday evening)

1. The cron job collects planned versus completed events for each domain
2. It lists slipped items and upcoming deadlines for the next 14 days
3. The agent drafts next week: recurring blocks first, then deadline work, then learning and blogging slots, then buffer
4. The review page shows the draft with Accept, Edit and Skip buttons
5. On Accept, the events are written and logged so the whole week can be undone in one step